claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a65ebb15c36eb8811.jsonl`
- SHA-256: `916a0a49c418138c5be312c7a982bf7dbc3cbf73f26e9af3ed4a7b46fb4446a4`
- Source modified: `2026-06-10T16:25:50+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`

Transcript

1. user (2026-06-10T16:22:08.054Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: Transient keychain read failure mints a new key and silently overwrites the only DB key
SEVERITY CLAIMED: high
FILE: src/muneem/db_key.py (line 24-31)
DESCRIPTION: `secrets.get_secret` (src/muneem/secrets.py:25-30) swallows every `KeyringError` and returns `None`, deliberately treating 'backend unavailable' the same as 'key absent'. `get_or_create_key()` then interprets that `None` as 'no key has ever been minted', generates a fresh key, and `set_secret` OVERWRITES the existing 'db-key' entry. If a read fails transiently (locked/temporarily failing Credential Manager session, keyring backend hiccup) while the subsequent write succeeds, the only copy of the real ledger key is destroyed; the next `connect_encrypted` fails with 'wrong key' and the data is gone unless a recovery file exists. `connect_encrypted` (src/muneem/db.py:207) calls `get_or_create_key()` unconditionally ? even when an encrypted ledger file already exists on disk, which is exactly the case where minting is never the right answer.
EVIDENCE QUOTED: def get_or_create_key() -> str:
    existing = _keychain.get_secret(_DB_KEY_NAME)
    if existing:
        return existing
    key = _stdlib_secrets.token_bytes(32).hex()
    _keychain.set_secret(_DB_KEY_NAME, key)
    return key
SUGGESTED FIX: Distinguish 'absent' from 'unavailable': add a get_secret variant that raises (or returns a sentinel) on KeyringError, and have get_or_create_key abort loudly instead of minting when the backend errored. Additionally, gate minting on the ledger file: if the encrypted DB file already exists but the keychain has no key, refuse with a pointer to 'muneem db recover' rather than minting a key that cannot open it.

Your job is to try to REFUTE it. Your lens: reachability and impact: check whether the scenario can actually occur in realistic single-user use on Windows, and whether the assigned severity is justified.
Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):
- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account

```
number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.
Structured output only.
```

2. user (2026-06-10T16:22:12.999Z)

```
1       """The ledger's encryption key ? minted once, kept in the OS keychain.
2
3       muneem's local ledger is **encrypted at rest** with SQLCipher (DESIGN.md §5.4 rule 4).
4       The 256-bit key is generated on first use and stored in the OS keychain via
5       :mod:`muneem.secrets` (Windows Credential Manager / macOS Keychain / Secret Service) ?
6       never on disk in plaintext, never in the repo, never logged.
7
8       It is handed to SQLCipher in **raw-key form** (``PRAGMA key = "x'<64 hex>'"``), which
uses
9       the 256-bit key directly and *skips* the passphrase KDF. That is correct here because
the
10      key is already full-entropy random; KDF stretching only buys security for low-entropy
human
11      passphrases (an opt-in we may add later ? see ROADMAP D.2 phased plan).
12      """
13
14      from __future__ import annotations
15
16      import secrets as _stdlib_secrets
17
18      from muneem import secrets as _keychain
19
20      # Keychain entry name (under the shared "muneem" service) holding the 64-hex-char DB
key.
21      _DB_KEY_NAME = "db-key"
22
23
24      def get_or_create_key() -> str:
25          """Return the ledger's 256-bit key as 64 hex chars, minting+storing one on first
use."""
26          existing = _keychain.get_secret(_DB_KEY_NAME)
27          if existing:
28              return existing
29          key = _stdlib_secrets.token_bytes(32).hex()
30          _keychain.set_secret(_DB_KEY_NAME, key)
31          return key
32
33
34      def peek_key() -> str | None:
35          """Return the stored key, or ``None`` if none has been minted yet. Never creates
one."""
36          return _keychain.get_secret(_DB_KEY_NAME)
37
38
39      def new_key() -> str:
40          """Generate a fresh random 256-bit key as 64 hex chars (not stored)."""
41          return _stdlib_secrets.token_bytes(32).hex()
42
43
44      def set_key(key: str) -> None:
45          """Overwrite the stored DB key ? used by key rotation, **after** the file has been
rekeyed."""
46          _keychain.set_secret(_DB_KEY_NAME, key)
47
```

3. user (2026-06-10T16:22:14.038Z)

```
1       """Secret storage via the OS keychain.
2
```

```
3       OAuth tokens, cached statement passwords, and the DB encryption key are stored here
(Windows
4       Credential Manager / macOS Keychain / Secret Service), never in the repo, on disk in
plaintext, or
5       in logs. See DESIGN.md section 5.
6
7       External-call discipline (every keyring call is wrapped with attribution):
8       - **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable backend returns
``None``
9         (absence is a valid state; this also keeps headless CI happy).
10      - **write** (:func:`set_secret`) is *loud* ? a failure raises
:class:`~muneem.errors.KeychainError`,
11        because silently losing a write (e.g. the DB key) would be dangerous. The secret
*name* is
12        included for diagnosis; the value never is.
13      """
14
15      from __future__ import annotations
16
17      import keyring
18      from keyring.errors import KeyringError, PasswordDeleteError
19
20      from muneem.errors import KeychainError
21
22      _SERVICE = "muneem"
23
24
25      def get_secret(name: str) -> str | None:
26          """Return the stored secret, or ``None`` if absent or the keychain is
unavailable."""
27          try:
28              return keyring.get_password(_SERVICE, name)
29          except KeyringError:
30              return None
31
32
33      def set_secret(name: str, value: str) -> None:
34          """Store (or replace) a secret in the OS keychain. Raises KeychainError on
failure."""
35          try:
36              keyring.set_password(_SERVICE, name, value)
37          except KeyringError as exc:
38              # Name only ? never the value ? in the message (it may be a password or the DB
key).
39              raise KeychainError(f"could not store secret {name!r} in the OS keychain") from
exc
40
41
42      def delete_secret(name: str) -> None:
43          """Delete a secret if present; a no-op if it is not (or the keychain is
unavailable)."""
44          try:
45              keyring.delete_password(_SERVICE, name)
46          except (PasswordDeleteError, KeyringError):
47              pass
48
```

4. user (2026-06-10T16:22:20.041Z)

```
1       """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.
2
3       This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4       (accounts, transactions, holdings, merchants, items) grows with later milestones.
5       ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
6       ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
```

```
    """

    from __future__ import annotations

    import os
    import sqlite3
    from collections.abc import Iterator
    from contextlib import contextmanager
    from pathlib import Path

    from muneem.errors import EncryptedDatabaseError  # re-exported so
db.EncryptedDatabaseError works

    SCHEMA_VERSION = 4

    # Unencrypted SQLite files start with this 16-byte magic header; an encrypted
(SQLCipher)
    # file does not, so we can tell them apart without a key.
    _SQLITE_MAGIC = b"SQLite format 3\x00"

    _SCHEMA = """
    CREATE TABLE IF NOT EXISTS documents (
        id            INTEGER PRIMARY KEY,
        source        TEXT NOT NULL,
        external_id   TEXT,
        sender        TEXT,
        subject       TEXT,
        received_date TEXT,
        doc_type      TEXT,
        sha256        TEXT NOT NULL UNIQUE,
        path          TEXT,
        status        TEXT NOT NULL DEFAULT 'parsed',
        created_at    TEXT NOT NULL DEFAULT (datetime('now'))
    );

    CREATE TABLE IF NOT EXISTS offers (
        id          INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        merchant    TEXT,
        promo_code  TEXT,
        description TEXT,
        expiry_raw  TEXT,
        expiry_type TEXT,
        source      TEXT NOT NULL DEFAULT 'text',
        redeemed    INTEGER NOT NULL DEFAULT 0,
        source_text TEXT
    );

    CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);

    -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
    -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
    -- Normalization into a unified view layer is planned for a later phase
    -- once all bank schemas are stabilized.
    CREATE TABLE IF NOT EXISTS axis_transactions (
        id INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        account_number TEXT,
        txn_date TEXT,
        value_date TEXT,
        particulars TEXT,
        debit REAL,
        credit REAL,
        balance REAL
    );
```

```
70
71      CREATE TABLE IF NOT EXISTS axis_cc_transactions (
72          id INTEGER PRIMARY KEY,
73          document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
74          account_number TEXT,
75          txn_date TEXT,
76          particulars TEXT,
77          merchant_category TEXT,
78          amount REAL,
79          type TEXT
80      );
81
82      CREATE TABLE IF NOT EXISTS au_transactions (
83          id INTEGER PRIMARY KEY,
84          document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
85          account_number TEXT,
86          txn_date TEXT,
87          value_date TEXT,
88          particulars TEXT,
89          debit REAL,
90          credit REAL,
91          balance REAL
92      );
93
94      CREATE TABLE IF NOT EXISTS hdfc_transactions (
95          id INTEGER PRIMARY KEY,
96          document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
97          account_number TEXT,
98          txn_date TEXT,
99          value_date TEXT,
100         particulars TEXT,
101         ref TEXT,
102         withdrawal REAL,
103         deposit REAL,
104         closing_balance REAL
105     );
106     """
107
108
109     def _raw_key_pragma(key: str) -> str:
110         """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF."""
111         return f"PRAGMA key = \"x'{key}'\""
112
113
114     def is_plaintext_sqlite(db_path: Path | str) -> bool:
115         """True iff the file exists and begins with the *unencrypted* SQLite magic header.
116
117         Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
118         without needing the key.
119         """
120         try:
121             with open(db_path, "rb") as fh:
122                 return fh.read(16) == _SQLITE_MAGIC
123         except OSError:
124             return False
125
126
127     def _harden_permissions(path: str) -> None:
128         """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
``:memory:``."""
129         if path == ":memory:":
130             return
131         try:
132             os.chmod(path, 0o600)
```

```
        except OSError:
            pass


    def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
        """Open an encrypted connection via SQLCipher, verifying the key before returning.

        A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
the
        key is wrong (or the file is not an encrypted muneem ledger), this raises.
        """
        try:
            import sqlcipher3
        except ImportError as exc:  # pragma: no cover - exercised only without the binding
            raise EncryptedDatabaseError(
                "database encryption requested but the 'sqlcipher3' package is not
installed"
            ) from exc

        conn = sqlcipher3.connect(path)
        conn.execute(_raw_key_pragma(key))  # MUST be the first statement on the connection
        try:
            conn.execute("SELECT count(*) FROM sqlite_master")  # forces key verification
        except sqlcipher3.DatabaseError as exc:
            conn.close()
            raise EncryptedDatabaseError(
                f"could not open encrypted database {Path(path).name}: wrong key, or not an
"
                "encrypted muneem ledger"
            ) from exc
        conn.row_factory = sqlcipher3.Row
        return conn


    @contextmanager
    def connect(db_path: Path | str, *, key: str | None = None) ->
Iterator[sqlite3.Connection]:
        """Open a connection (commit on success, always close). Parent dirs created.

        With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
        SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
path for
        ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
        :func:`_open_encrypted`); the production ledger is opened via
:func:`connect_encrypted`.
        """
        path = str(db_path)
        if path != ":memory:":
            Path(path).parent.mkdir(parents=True, exist_ok=True)
        if key is not None:
            conn = _open_encrypted(path, key)
            _harden_permissions(path)
        else:
            conn = sqlite3.connect(path)
            conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA foreign_keys = ON")
        try:
            yield conn
            conn.commit()
        finally:
            conn.close()


    @contextmanager
    def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
```

```
        """Open the real on-disk ledger, encrypted with the keychain-managed key.

        The production path: resolves (or mints, on first use) the 256-bit key from the OS
        keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
SQLite
        file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
        "file is not a database" error.
        """
        from muneem.db_key import get_or_create_key

        path = str(db_path)
        if is_plaintext_sqlite(path):
            raise EncryptedDatabaseError(
                f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
                "to migrate it to an encrypted ledger"
            )
        with connect(path, key=get_or_create_key()) as conn:
            yield conn


    def encrypt_database(db_path: Path | str, key: str) -> int:
        """Migrate a plaintext SQLite ledger to an encrypted one in place, via
``sqlcipher_export``.

        The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
then
        securely delete (it still holds the cleartext data). Returns the number of user
tables
        copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
SQLite file.
        """
        import sqlcipher3

        path = Path(db_path)
        if not is_plaintext_sqlite(path):
            raise EncryptedDatabaseError(
                f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
            )

        enc_tmp = path.with_name(path.name + ".enc.tmp")
        enc_tmp.unlink(missing_ok=True)
        try:
            src = sqlcipher3.connect(str(path))  # open the plaintext source (no key)
            try:
                src_tables = src.execute(
                    "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
                ).fetchone()[0]
                target = str(enc_tmp).replace("\\", "/")  # SQL string literal: forward
slashes
                src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY \"x'{key}'\"")
                src.execute("SELECT sqlcipher_export('encrypted')")
                src.execute("DETACH DATABASE encrypted")
            finally:
                src.close()

            # Verify the encrypted copy BEFORE the destructive swap: it must open with the
key, pass
            # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
            # so a failed check is recoverable ? but we refuse to swap in a bad file.)
            with connect(str(enc_tmp), key=key) as check:
                integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
                ntables = check.execute(
```

```
                    "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
                ).fetchone()[0]
            if integrity != "ok" or ntables != src_tables:
                raise EncryptedDatabaseError(
                    f"encryption migration verification failed for {path.name} "
                    f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original
left untouched"
                )

            backup = path.with_name(path.name + ".plaintext.bak")
            backup.unlink(missing_ok=True)
            path.rename(backup)
            enc_tmp.rename(path)  # consumed on success
            _harden_permissions(str(path))
        finally:
            # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
            enc_tmp.unlink(missing_ok=True)
        return int(ntables)


    def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
        """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).

        Opens with ``current_key`` first (which verifies it), then rekeys. Raises
        :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
        muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
        after this returns successfully.
        """
        conn = _open_encrypted(str(db_path), current_key)  # validates current_key
        try:
            conn.execute(f"PRAGMA rekey = \"x'{new_key}'\"")
            conn.commit()
        finally:
            conn.close()
        _harden_permissions(str(db_path))


    def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
        """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).

        Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
        naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
        it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
        """
        import sqlcipher3

        dest_path = Path(dest)
        dest_path.parent.mkdir(parents=True, exist_ok=True)
        dest_path.unlink(missing_ok=True)
        source = _open_encrypted(str(db_path), key)  # validates key
        try:
            target = sqlcipher3.connect(str(dest_path))
            try:
                target.execute(_raw_key_pragma(key))  # key the destination so the backup is
encrypted
                source.backup(target)  # consistent snapshot of the live DB
            finally:
                target.close()
        finally:
```

```
304             source.close()
305         _harden_permissions(str(dest_path))
306 
307 
308     def init_db(conn: sqlite3.Connection) -> None:
309         """Bring the schema up to date via the migration runner (creates tables on a fresh
DB).
310 
311         The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
future changes
312         are forward migrations in :mod:`muneem.migrations`, recorded in
``schema_migrations``.
313         """
314         from muneem import migrations
315 
316         migrations.run_migrations(conn, baseline_sql=_SCHEMA,
baseline_version=SCHEMA_VERSION)
317 
318 
319     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
320         return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
(sha256,)).fetchone()
321 
322 
323     def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
324         """Internal helper for the simple INSERT pattern used by documents, offers, and bank
txns."""
325         cols = ", ".join(fields)
326         placeholders = ", ".join("?" for _ in fields)
327         cur = conn.execute(
328             f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
329             tuple(fields.values()),
330         )
331         rowid = cur.lastrowid  # always set after a successful INSERT; guard for the type-
checker
332         if rowid is None:  # pragma: no cover - defensive; INSERT always sets lastrowid
333             raise RuntimeError(f"INSERT into {table} did not return a row id")
334         return int(rowid)
335 
336 
337     def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
338         """Insert into documents table. Delegates to the shared helper for DRY."""
339         return _insert_row(conn, "documents", dict(fields))
340 
341 
342     def record_provenance(
343         conn: sqlite3.Connection,
344         *,
345         document_id: int,
346         source_type: str,
347         parser: str | None = None,
348         parser_version: str | None = None,
349         mapping_layer: str | None = None,
350         confidence: str | None = None,
351     ) -> None:
352         """Upsert the provenance row for a parse (one per document ? see DESIGN §8.7).
353 
354         First-class but *not* per transaction row: every txn table FKs ``documents(id)``, so
a single
355         provenance record per document is reachable by join. Idempotent on ``document_id``
(a re-parse
356         overwrites rather than duplicating).
357         """
358         conn.execute(
359             "INSERT INTO provenance "
```

```
            "(document_id, source_type, parser, parser_version, mapping_layer, confidence) "
            "VALUES (?, ?, ?, ?, ?, ?) "
            "ON CONFLICT(document_id) DO UPDATE SET "
            "source_type=excluded.source_type, parser=excluded.parser, "
            "parser_version=excluded.parser_version, mapping_layer=excluded.mapping_layer, "
            "confidence=excluded.confidence",
            (document_id, source_type, parser, parser_version, mapping_layer, confidence),
        )


    def record_parse(
        conn: sqlite3.Connection,
        document_id: int,
        *,
        confidence: str,
        parser: str,
        source_type: str = "pdf_statement",
        parser_version: str | None = None,
        mapping_layer: str = "deterministic",
    ) -> None:
        """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
parse)."""
        conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence,
document_id))
        record_provenance(
            conn,
            document_id=document_id,
            source_type=source_type,
            parser=parser,
            parser_version=parser_version,
            mapping_layer=mapping_layer,
            confidence=confidence,
        )


    def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
        return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
(document_id,)).fetchone()


    def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "offers", fields)


    def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
        return conn.execute(
            """
            SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
                   d.external_id, d.path
            FROM offers o
            JOIN documents d ON d.id = o.document_id
            ORDER BY o.id
            """
        ).fetchall()


    def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "axis_transactions", fields)


    def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "axis_cc_transactions", fields)


    def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
```

```
list[sqlite3.Row]:
        return conn.execute(
            "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
        ).fetchall()


    def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
        return conn.execute(
            "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
        ).fetchall()


    def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "hdfc_transactions", fields)


    def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
        return conn.execute(
            "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
        ).fetchall()


    def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "au_transactions", fields)


    def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
        return conn.execute(
            "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
        ).fetchall()

```

5. user (2026-06-10T16:22:53.577Z)

```
src\muneem\__init__.py
src\muneem\config.py
src\muneem\offers.py
src\muneem\ocr.py
src\muneem\extract.py
src\muneem\log.py
src\muneem\redaction.py
src\muneem\password_rules.py
src\muneem\secrets.py
src\muneem\statement_unlock.py
src\muneem\unlock.py
src\muneem\db_key.py
src\muneem\cli.py
src\muneem\recovery.py
src\muneem\errors.py
src\muneem\db.py
src\muneem\migrations.py
```

6. user (2026-06-10T16:22:54.654Z)

```
Found 7 files
src\muneem\db.py
src\muneem\parsers\hdfc.py
src\muneem\recovery.py
src\muneem\cli.py
src\muneem\db_key.py
src\muneem\secrets.py
src\muneem\parsers\savings.py
```

7. user (2026-06-10T16:22:58.630Z)

```
1       """Disaster recovery for the encrypted ledger ? a passphrase-wrapped recovery file.
2
3       The DB key lives in the OS keychain (fast, full-entropy). If that keychain is ever lost
? OS
4       reinstall, new machine, profile/Credential-Manager corruption ? the ledger **and every
same-key
5       backup** become permanently unrecoverable. This module lets the user wrap that key under
a
6       passphrase they choose, into a recovery file they back up **off-machine**; with the file
*and* the
7       passphrase
8       they can restore the key into a fresh keychain.
9
10      Crypto: **Argon2id** (memory-hard KDF) derives a 256-bit wrapping key from the
passphrase + a random
11      salt; **AES-256-GCM** (authenticated) wraps the DB key. The recovery file holds only
12      ``{version, kdf params, salt, nonce, ciphertext}`` ? never the key, never the
passphrase. A wrong
13      passphrase fails the GCM authentication tag ? a clean error, with no decryption oracle.
14      """
15
16      from __future__ import annotations
17
18      import base64
19      import json
20      import os
21      from pathlib import Path
22      from typing import Any
23
24      from cryptography.exceptions import InvalidTag
25      from cryptography.hazmat.primitives.ciphers.aead import AESGCM
26      from cryptography.hazmat.primitives.kdf.argon2 import Argon2id
27
28      from muneem.errors import MuneemError
29
30      RECOVERY_VERSION = 1
31      _AAD = b"muneem-recovery-v1"  # binds the ciphertext to this format/version
32
33      # Argon2id parameters (memory in KiB). Conservative-but-real; the file stores them so
they can be
34      # raised over time without breaking older recovery files.
35      _TIME_COST = 3
36      _MEMORY_KIB = 64 * 1024  # 64 MiB
37      _LANES = 4
38      _SALT_BYTES = 16
39      _NONCE_BYTES = 12
40      _KEY_BYTES = 32
41      _MIN_PASSPHRASE = 8
42
43
44      class RecoveryError(MuneemError):
45          """A recovery file could not be created, or could not be opened (wrong passphrase /
corrupt)."""
46
47
48      def _derive(passphrase: str, salt: bytes, *, time_cost: int, memory_kib: int, lanes:
int) -> bytes:
49          kdf = Argon2id(
50              salt=salt,
51              length=_KEY_BYTES,
52              iterations=time_cost,
53              lanes=lanes,
54              memory_cost=memory_kib,
```

```
        )
        return kdf.derive(passphrase.encode("utf-8"))


    def wrap_key(db_key: str, passphrase: str) -> dict[str, Any]:
        """Return a JSON-serialisable recovery payload wrapping ``db_key`` under
``passphrase``."""
        if not passphrase or len(passphrase) < _MIN_PASSPHRASE:
            raise RecoveryError(f"recovery passphrase must be at least {_MIN_PASSPHRASE}
characters")
        salt = os.urandom(_SALT_BYTES)
        nonce = os.urandom(_NONCE_BYTES)
        key = _derive(passphrase, salt, time_cost=_TIME_COST, memory_kib=_MEMORY_KIB,
lanes=_LANES)
        ciphertext = AESGCM(key).encrypt(nonce, db_key.encode("utf-8"), _AAD)
        return {
            "version": RECOVERY_VERSION,
            "kdf": {
                "algo": "argon2id",
                "time_cost": _TIME_COST,
                "memory_kib": _MEMORY_KIB,
                "lanes": _LANES,
            },
            "salt": base64.b64encode(salt).decode(),
            "nonce": base64.b64encode(nonce).decode(),
            "ciphertext": base64.b64encode(ciphertext).decode(),
        }


    def unwrap_key(payload: dict[str, Any], passphrase: str) -> str:
        """Recover the DB key from a recovery ``payload`` + ``passphrase``. Raises
RecoveryError."""
        try:
            salt = base64.b64decode(payload["salt"])
            nonce = base64.b64decode(payload["nonce"])
            ciphertext = base64.b64decode(payload["ciphertext"])
            kdf = payload.get("kdf", {})
        except (KeyError, ValueError, TypeError) as exc:
            raise RecoveryError("recovery file is malformed") from exc
        key = _derive(
            passphrase,
            salt,
            time_cost=int(kdf.get("time_cost", _TIME_COST)),
            memory_kib=int(kdf.get("memory_kib", _MEMORY_KIB)),
            lanes=int(kdf.get("lanes", _LANES)),
        )
        try:
            plain = AESGCM(key).decrypt(nonce, ciphertext, _AAD)
        except InvalidTag as exc:
            raise RecoveryError("wrong passphrase or corrupt recovery file") from exc
        return plain.decode("utf-8")


    def write_recovery_file(path: Path | str, db_key: str, passphrase: str) -> Path:
        """Wrap ``db_key`` under ``passphrase`` and write the recovery file (owner-only
perms)."""
        payload = wrap_key(db_key, passphrase)
        out = Path(path)
        out.parent.mkdir(parents=True, exist_ok=True)
        out.write_text(json.dumps(payload, indent=2), encoding="utf-8")
        try:
            os.chmod(out, 0o600)
        except OSError:  # pragma: no cover - best-effort on platforms without POSIX perms
            pass
        return out
```

```
115
116
117     def read_recovery_file(path: Path | str, passphrase: str) -> str:
118         """Read a recovery file and return the unwrapped DB key. Raises RecoveryError on any
failure."""
119         p = Path(path)
120         try:
121             payload = json.loads(p.read_text(encoding="utf-8"))
122         except (OSError, ValueError) as exc:
123             raise RecoveryError(f"could not read recovery file {p.name}") from exc
124         return unwrap_key(payload, passphrase)
125
```

8. user (2026-06-10T16:22:59.932Z)

```
44:        with dbmod.connect_encrypted(db_path) as conn:
393:    from muneem.db_key import get_or_create_key
406:    n = dbmod.encrypt_database(db_path, get_or_create_key())
468:@db_app.command("setup-recovery")
469:def db_setup_recovery(
471:        None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
474:    """Create a passphrase-wrapped recovery file for the ledger's encryption key.
481:    from muneem import recovery
488:    out_path = out or (load_config().data_dir / "muneem-recovery.json")
489:    passphrase = getpass.getpass("New recovery passphrase: ")
494:        recovery.write_recovery_file(out_path, key, passphrase)
495:    except recovery.RecoveryError as exc:
499:        f"wrote recovery file to {out_path}. Store it OFF this machine (password manager / "
500:        "separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a "
501:        "lost keychain means the ledger and its backups are unrecoverable."
505:@db_app.command("recover")
506:def db_recover(
510:    """Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain)."""
514:    from muneem import db_key, recovery
518:        key = recovery.read_recovery_file(file, passphrase)
519:    except recovery.RecoveryError as exc:
523:    # If the ledger is present, verify the recovered key actually opens it before committing
it.
529:            typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
532:    typer.echo("recovered the encryption key into the keychain.")
```

9. user (2026-06-10T16:23:07.180Z)

```
.venv\Lib\site-packages\keyring\backends\Windows.py
```

10. user (2026-06-10T16:23:10.802Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import keyring; print(keyring.__version__); print(keyring.get_keyring())
                          ^^^^^^^^^^^^^^^^^^^
AttributeError: module 'keyring' has no attribute '__version__'
Name
----
.venv
```

11. user (2026-06-10T16:23:16.014Z)

```
1       from __future__ import annotations
2
```

```
    import logging

    from jaraco.context import ExceptionTrap

    from ..backend import KeyringBackend
    from ..compat import properties
    from ..credentials import SimpleCredential
    from ..errors import PasswordDeleteError

    with ExceptionTrap() as missing_deps:
        try:
            # prefer pywin32-ctypes
            from win32ctypes.pywin32 import pywintypes, win32cred

            # force demand import to raise ImportError
            win32cred.__name__  # noqa: B018
        except ImportError:
            # fallback to pywin32
            import pywintypes
            import win32cred

            # force demand import to raise ImportError
            win32cred.__name__  # noqa: B018

    log = logging.getLogger(__name__)


    class Persistence:
        def __get__(self, keyring, type=None):
            return getattr(keyring, '_persist', win32cred.CRED_PERSIST_ENTERPRISE)

        def __set__(self, keyring, value):
            """
            Set the persistence value on the Keyring. Value may be
            one of the win32cred.CRED_PERSIST_* constants or a
            string representing one of those constants. For example,
            'local machine' or 'session'.
            """
            if isinstance(value, str):
                attr = 'CRED_PERSIST_' + value.replace(' ', '_').upper()
                value = getattr(win32cred, attr)
            keyring._persist = value


    class DecodingCredential(dict):
        @property
        def value(self):
            """
            Attempt to decode the credential blob as UTF-16 then UTF-8.
            """
            cred = self['CredentialBlob']
            try:
                return cred.decode('utf-16')
            except UnicodeDecodeError:
                decoded_cred_utf8 = cred.decode('utf-8')
                log.warning(
                    "Retrieved a UTF-8 encoded credential. Please be aware that "
                    "this library only writes credentials in UTF-16."
                )
                return decoded_cred_utf8


    class WinVaultKeyring(KeyringBackend):
        """
        WinVaultKeyring stores encrypted passwords using the Windows Credential
```

```
        Manager.

        Requires pywin32

        This backend does some gymnastics to simulate multi-user support,
        which WinVault doesn't support natively. See
        https://github.com/jaraco/keyring/issues/47#issuecomment-75763152
        for details on the implementation, but here's the gist:

        Passwords are stored under the service name unless there is a collision
        (another password with the same service name but different user name),
        in which case the previous password is moved into a compound name:
        {username}@{service}
        """

        persist = Persistence()

        @properties.classproperty
        def priority(cls) -> float:
            """
            If available, the preferred backend on Windows.
            """
            if missing_deps:
                raise RuntimeError("Requires Windows and pywin32")
            return 5

        @staticmethod
        def _compound_name(username, service):
            return f'{username}@{service}'

        def get_password(self, service, username):
            res = self._resolve_credential(service, username)
            return res and res.value

        def _resolve_credential(
            self, service: str, username: str | None
        ) -> DecodingCredential | None:
            # first attempt to get the password under the service name
            res = self._read_credential(service)
            if not res or username and res['UserName'] != username:
                # It wasn't found so attempt to get it with the compound name
                res = self._read_credential(self._compound_name(username, service))
            return res

        def _read_credential(self, target):
            try:
                res = win32cred.CredRead(
                    Type=win32cred.CRED_TYPE_GENERIC, TargetName=target
                )
            except pywintypes.error as e:
                if e.winerror == 1168 and e.funcname == 'CredRead':  # not found
                    return None
                raise
            return DecodingCredential(res)

        def set_password(self, service, username, password):
            existing_pw = self._read_credential(service)
            if existing_pw:
                # resave the existing password using a compound target
                existing_username = existing_pw['UserName']
                target = self._compound_name(existing_username, service)
                self._set_password(
                    target,
                    existing_username,
                    existing_pw.value,
```

```
                )
            self._set_password(service, username, str(password))

        def _set_password(self, target, username, password):
            credential = dict(
                Type=win32cred.CRED_TYPE_GENERIC,
                TargetName=target,
                UserName=username,
                CredentialBlob=password,
                Comment="Stored using python-keyring",
                Persist=self.persist,
            )
            win32cred.CredWrite(credential, 0)

        def delete_password(self, service, username):
            compound = self._compound_name(username, service)
            deleted = False
            for target in service, compound:
                existing_pw = self._read_credential(target)
                if existing_pw and existing_pw['UserName'] == username:
                    deleted = True
                    self._delete_password(target)
            if not deleted:
                raise PasswordDeleteError(service)

        def _delete_password(self, target):
            try:
                win32cred.CredDelete(Type=win32cred.CRED_TYPE_GENERIC, TargetName=target)
            except pywintypes.error as e:
                if e.winerror == 1168 and e.funcname == 'CredDelete':  # not found
                    return
                raise

        def get_credential(self, service, username):
            res = self._resolve_credential(service, username)
            return res and SimpleCredential(res['UserName'], res.value)

```

12. user (2026-06-10T16:23:18.767Z)

```
25.7.0
keyring.backends.Windows.WinVaultKeyring (priority: 5)
```

13. user (2026-06-10T16:23:57.305Z)

```
27-    if not isinstance(keyring, backend.KeyringBackend):
28:        raise TypeError("The keyring must be an instance of KeyringBackend")
29-    _keyring_backend = keyring
30-
31-
--
52-        os.makedirs(root)
53:    except OSError:
54-        pass
55-    filename = os.path.join(root, 'keyringrc.cfg')
56-    if os.path.exists(filename):
57-        msg = f"Refusing to overwrite {filename}"
58:        raise RuntimeError(msg)
59-    with open(filename, 'w', encoding='utf-8') as file:
60-        file.write('[backend]\ndefault-keyring=keyring.backends.null.Keyring')
61-
62-
63:def get_password(service_name: str, username: str) -> str | None:
64-    """Get password from the specified service."""
65-    return get_keyring().get_password(service_name, username)
```

```
66-
67-
68:def set_password(service_name: str, username: str, password: str) -> None:
69-    """Set password for the user in the specified service."""
70-    get_keyring().set_password(service_name, username, password)
71-
--
144-    class_ = _load_keyring_class(keyring_name)
145:    # invoke the priority to ensure it is viable, or raise a RuntimeError
146-    class_.priority  # noqa: B018
147-    return class_()
148-
--
153-        return load_keyring(os.environ['PYTHON_KEYRING_BACKEND'])
154:    except KeyError:
155-        return None
156-
157-
--
163-    if not path.exists():
164:        raise FileNotFoundError(path)
165-    return path
166-
167-
--
173-        config.read(_ensure_path(_config_path()), encoding='utf-8')
174:    except FileNotFoundError:
175-        return None
176-    _load_keyring_path(config)
177-
--
184-
185:    except (configparser.NoOptionError, ImportError):
186-        logger = logging.getLogger('keyring')
187-        logger.warning(
188-            "Keyring config file contains incorrect values.\n"
--
200-        sys.path.insert(0, os.path.expanduser(path))
201:    except (configparser.NoOptionError, configparser.NoSectionError):
202-        pass
```

14. user (2026-06-10T16:23:58.303Z)

```
No matches found
```

15. user (2026-06-10T16:24:04.836Z)

```
15:class error(Exception):
16-    def __init__(self, *args, **kw):
17-        nargs = len(args)
18-        if nargs > 0:
19:            self.winerror = args[0]
20-        else:
21:            self.winerror = None
22-        if nargs > 1:
23-            self.funcname = args[1]
24-        else:
25-            self.funcname = None
26-        if nargs > 2:
27:            self.strerror = args[2]
28-        else:
29:            self.strerror = None
30-        Exception.__init__(self, *args, **kw)
31-
32-
```

```
33-@contextlib.contextmanager
34:def pywin32error():
35-    try:
36-        yield
37-    except WindowsError as exception:
38-        if not hasattr(exception, 'function'):
39-            exception.function = 'unknown'
40:        raise error(exception.winerror, exception.function, exception.strerror)
41-
42-
43-class datetime(_datetime):
44-
45-    def Format(self, fmt='%c'):
46-        return self.strftime(fmt)
--
63:        except OSError as error:
64:            if error.errno == 22:
65:                raise ValueError(error.strerror)
66-            else:
67-                raise
```

16. user (2026-06-10T16:25:04.586Z)

```
1       import logging
2       from contextlib import closing
3
4       from jaraco.context import ExceptionTrap
5
6       from .. import backend
7       from ..backend import KeyringBackend
8       from ..compat import properties
9       from ..credentials import SimpleCredential
10      from ..errors import (
11          InitError,
12          KeyringLocked,
13          PasswordDeleteError,
14      )
15
16      try:
17          import secretstorage
18          import secretstorage.exceptions as exceptions
19      except ImportError:
20          pass
21      except AttributeError:
22          # See https://github.com/jaraco/keyring/issues/296
23          pass
24
25      log = logging.getLogger(__name__)
26
27
28      class Keyring(backend.SchemeSelectable, KeyringBackend):
29          """Secret Service Keyring"""
30
31          appid = 'Python keyring library'
32
33          @properties.classproperty
34          def priority(cls) -> float:
35              with ExceptionTrap() as exc:
36                  secretstorage.__name__  # noqa: B018
37              if exc:
38                  raise RuntimeError("SecretStorage required")
39              if secretstorage.__version_tuple__ < (3, 2):
40                  raise RuntimeError("SecretStorage 3.2 or newer required")
41              try:
42                  with closing(secretstorage.dbus_init()) as connection:
```

```
                    if not secretstorage.check_service_availability(connection):
                        raise RuntimeError(
                            "The Secret Service daemon is neither running nor "
                            "activatable through D-Bus"
                        )
            except exceptions.SecretStorageException as e:
                raise RuntimeError(f"Unable to initialize SecretService: {e}") from e
            return 5

        def get_preferred_collection(self):
            """If self.preferred_collection contains a D-Bus path,
            the collection at that address is returned. Otherwise,
            the default collection is returned.
            """
            bus = secretstorage.dbus_init()
            try:
                if hasattr(self, 'preferred_collection'):
                    collection = secretstorage.Collection(bus, self.preferred_collection)
                else:
                    collection = secretstorage.get_default_collection(bus)
            except exceptions.SecretStorageException as e:
                raise InitError(f"Failed to create the collection: {e}.") from e
            if collection.is_locked():
                collection.unlock()
                if collection.is_locked():  # User dismissed the prompt
                    raise KeyringLocked("Failed to unlock the collection!")
            return collection

        def unlock(self, item):
            if hasattr(item, 'unlock'):
                item.unlock()
            if item.is_locked():  # User dismissed the prompt
                raise KeyringLocked('Failed to unlock the item!')

        def get_password(self, service, username):
            """Get password of the username for the service"""
            collection = self.get_preferred_collection()
            with closing(collection.connection):
                items = collection.search_items(self._query(service, username))
                for item in items:
                    self.unlock(item)
                    return item.get_secret().decode('utf-8')

        def set_password(self, service, username, password):
            """Set password for the username of the service"""
            collection = self.get_preferred_collection()
            attributes = self._query(service, username, application=self.appid)
            label = f"Password for '{username}' on '{service}'"
            with closing(collection.connection):
                collection.create_item(label, attributes, password, replace=True)

        def delete_password(self, service, username):
            """Delete the stored password (only the first one)"""
            collection = self.get_preferred_collection()
            with closing(collection.connection):
                items = collection.search_items(self._query(service, username))
                for item in items:
                    return item.delete()
            raise PasswordDeleteError("No such password!")

        def get_credential(self, service, username):
            """Gets the first username and password for a service.
            Returns a Credential instance

            The username can be omitted, but if there is one, it will use get_password
```

```
108             and return a SimpleCredential containing  the username and password
109             Otherwise, it will return the first username and password combo that it finds.
110             """
111             scheme = self.schemes[self.scheme]
112             query = self._query(service, username)
113             collection = self.get_preferred_collection()
114
115             with closing(collection.connection):
116                 items = collection.search_items(query)
117                 for item in items:
118                     self.unlock(item)
119                     username = item.get_attributes().get(scheme['username'])
120                     return SimpleCredential(username, item.get_secret().decode('utf-8'))
```

17. user (2026-06-10T16:25:06.638Z)

```
5:class KeyringError(Exception):
6-    """Base class for exceptions in keyring"""
7-
--
17:class InitError(KeyringError):
18-    """Raised when the keyring could not be initialised"""
19-
--
21:class KeyringLocked(KeyringError):
22-    """Raised when the keyring failed unlocking"""
23-
--
25:class NoKeyringError(KeyringError, RuntimeError):
26-    """Raised when there is no keyring backend"""
27-
```

18. user (2026-06-10T16:25:46.572Z)

```
Structured output provided successfully
```

19. assistant (2026-06-10T16:25:50.248Z)

```
Verification complete.
```